[1부 전용] Windows (WSL2) + Docker + OpenClaw + Gemini 셋업 가이드

> 이 문서는 1부 강의에서 도커 설치부터 Gemini API 키까지의 OpenClaw 전체 셋업 과정에 어려움을 겪는 분들을 위한 가이드 문서입니다. 이 문서는 **1부 수강생**이 Windows + WSL2 + Docker 환경에서 **OpenClaw 웹 대시보드에 접속하고, 기기 페어링까지 완료한 뒤, 채팅창에서 첫 인사 메시지를 주고받는 것**을 목표로 한다.
> **토큰 비밀번호는 `.env` 파일에, Gemini API 키는 온보딩 과정에서 직접 붙여넣어 `auth-profiles.json`에 키가 그대로 저장되는 방식**으로 진행한다.

---

0. 전체 흐름 한눈에 보기

1. Docker Desktop 설치 (윈도우 샌드박스 준비)
2. WSL2 + Ubuntu 설치 및 Docker 연동
3. Ubuntu 안에 OpenClaw 프로젝트 폴더 만들기
4. `.env` 파일에 토큰/경로/포트 설정하기 (1부 전용)
5. `docker-compose.yml` 작성 및 OpenClaw 컨테이너 실행
6. `openclaw.json` 최소 설정 추가
7. 온보딩(onboard CLI)으로 Gemini 연결
8. 브라우저에서 대시보드 접속 → 기기 페어링 → 채팅창에서 첫 인사

---

1. Docker Desktop 설치 (Windows)

1-1. 사전 확인

- RAM: 8GB 이상 (16GB 권장)
- 디스크 여유 공간: 최소 5GB 이상
- Windows 11 권장

1-2. 설치

1. 브라우저에서 Docker 공식 사이트 접속 → Download Docker Desktop.
2. Windows용 `.exe` 설치 파일 다운로드.
3. 설치 시 **Use WSL 2 instead of Hyper-V (recommended)** 체크.
4. 설치 완료 후 **Close and restart**로 PC 재부팅.
5. Docker Desktop 첫 실행 시
    - 로그인 화면: **Continue without signing in**.
    - 설문: **Skip**.

1-3. 설치 확인

PowerShell 또는 CMD에서:

```
docker --version
docker run hello-world
```

`Hello from Docker!` 메시지가 나오면 설치 성공.

---

2. WSL2 + Ubuntu 설치 및 Docker 연동

2-1. WSL2 + Ubuntu 설치

1. 관리자 권한 PowerShell 실행.
2. 다음 명령 입력:

```
wsl --install
```

3. 재부팅 후 자동으로 뜨는 Ubuntu 터미널에서 username / password 설정.

2-2. Docker Desktop ↔ WSL2 연동

1. Docker Desktop 실행.
2. Settings → General
   - **Use the WSL 2 based engine** 체크.
3. Settings → Resources → WSL Integration
   - **Enable integration with my default WSL distro** ON.
   - 목록에서 **Ubuntu** 토글 ON → Apply & Restart.
4. Ubuntu 터미널에서:

```
docker --version
docker compose version
```

둘 다 버전 정보가 나오면 연동 완료.

---

3. Ubuntu 내부 프로젝트 폴더 만들기

> 반드시 **Ubuntu 홈 디렉터리(~)** 아래에 만들 것.
> `/mnt/c`, `/mnt/d` 등 Windows 드라이브를 바로 마운트하면 권한 문제(EPERM)가 잘 난다.

```
# 1. 홈 디렉터리 이동
cd ~

# 2. 프로젝트 및 하위 폴더 생성
mkdir -p ~/demo-p1/config ~/demo-p1/workspace

# 3. 작업 디렉터리로 이동
cd ~/demo-p1
```

이 폴더(`~/demo-p1`)가 1부 실습의 "루트 프로젝트 폴더"가 된다.

---

4. .env 파일에 토큰/경로/포트 설정 (1부 방식)

> 1부에서는 초반에는 **토큰·경로·포트 등을 `.env` 파일로 관리**하고,
> **Gemini API 키는 온보딩 과정에서 직접 붙여넣어 `auth-profiles.json`에 기록**한다.

4-1. .env 파일 생성

Ubuntu에서:

```
cd ~/demo-p1
nano .env
```

아래 내용을 붙여넣고, 필요한 부분만 수정한다.

```
# 게이트웨이 토큰 (임의의 길고 예측 어려운 문자열 추천)
OPENCLAW_GATEWAY_TOKEN=your_secure_token_here

# 게이트웨이 모드/포트
OPENCLAW_GATEWAY_MODE=local
OPENCLAW_GATEWAY_PORT=3000

# 호스트(우분투) 기준 설정/워크스페이스 경로
OPENCLAW_CONFIG_DIR=./config
OPENCLAW_WORKSPACE_DIR=./workspace
```

- `OPENCLAW_GATEWAY_TOKEN` 값은 충분히 길고 랜덤하게 작성.
- `.env` 파일은 Git에 올리지 않는 것을 권장.

저장 후 종료(Ctrl+O, Enter, Ctrl+X).

---

5. docker-compose.yml 작성

```
cd ~/demo-p1
nano docker-compose.yml
```

아래 예시를 붙여넣는다.

```
services:
  openclaw-gateway:
    image: ghcr.io/openclaw/openclaw:latest
    container_name: openclaw-gateway
    env_file:
      - .env
    environment:
      - NODE_ENV=production
      - OPENCLAW_GATEWAY_TOKEN=${OPENCLAW_GATEWAY_TOKEN}
      - OPENCLAW_GATEWAY_MODE=${OPENCLAW_GATEWAY_MODE}
    volumes:
      - ${OPENCLAW_CONFIG_DIR}:/home/node/.openclaw
      - ${OPENCLAW_WORKSPACE_DIR}:/home/node/workspace
    ports:
      - "${OPENCLAW_GATEWAY_PORT}:18789"
    restart: unless-stopped
    command: [
      "node", "dist/index.js",
      "gateway",
      "--allow-unconfigured",
      "--bind", "lan",
      "--port", "18789",
      "--token", "${OPENCLAW_GATEWAY_TOKEN}"
    ]
```

포인트:

- 토큰/포트/경로는 `.env`에서 읽어온다.
- 컨테이너 내부 경로는 `/home/node/.openclaw`, `/home/node/workspace`로 고정.

---

6. 최소 설정 파일(openclaw.json) 추가

```
cd ~/demo-p1/config
nano openclaw.json
```

내용:

```
{
  "gateway": {
    "mode": "local",
```

```
      "bind": "loopback",
      "controlUi": {
        "allowedOrigins": [
          "http://localhost:3000",
          "http://127.0.0.1:3000"
        ]
      }
    }
  }
```

- `mode: "local"`: 로컬 사용 모드.
- `bind: "loopback"`: 로컬호스트에만 바인딩.
- `allowedOrigins`: 웹 대시보드가 접속할 주소.

---

7. 컨테이너 실행 및 온보딩

7-1. 컨테이너 실행

```
cd ~/demo-p1

docker compose up -d

docker compose logs -f   # 선택: 로그 확인
```

에러 없이 올라가면 게이트웨이 컨테이너 기동 완료.

7-2. 온보딩(onboard)으로 Gemini 연결 (1부 방식)

> 여기서 **Gemini API 키를 복사해서 직접 입력하면**,
> 컨테이너 내부의 `auth-profiles.json` 파일에 **키가 그대로 저장되는 1부 방식**으로 동작한다.

```
cd ~/demo-p1
docker compose exec openclaw-gateway node dist/index.js onboard
```

온보딩 흐름에서:

- Onboarding mode
  - **QuickStart (Configure details later via openclaw configure.)**
- Config handling
  - **Update values**
- Model/auth provider
  - **Google (Gemini API key + OAuth)**
- Google auth method
  - **Google Gemini API key**
- How do you want to provide this API key?

    - **Paste API key now** 선택 →
      Google AI Studio에서 발급받은 **실제 Gemini API 키 문자열을 그대로 붙여넣기**.
- Default model
    - **google/gemini-2.5-flash** (또는 강의에서 지정한 모델)

마지막에 `Onboarding complete` 메시지가 나오면 완료.

> 이 시점에서 컨테이너 내부 `auth-profiles.json`에는
> `google_api_key`가 **평문으로 저장되어 있는 상태**이며,
> 1부에서는 이 형태를 그대로 사용한다.

---

8. 대시보드 접속 → 페어링 → 첫 인사

8-1. 대시보드 접속

브라우저(Chrome 등)에서:

```
http://localhost:3000?token=YOUR_TOKEN_HERE
```

- `YOUR_TOKEN_HERE` 자리에 `.env`에 적은 `OPENCLAW_GATEWAY_TOKEN` 값을 그대로 넣는다.
- 화면에 **Pairing Required** 또는 비슷한 안내가 뜨면,
    - **포트/토큰/게이트웨이 통신**까지는 정상.

8-2. 기기 페어링

Ubuntu에서:

```
cd ~/demo-p1

# 현재 대기 중인 디바이스 목록 확인
docker compose exec openclaw-gateway node dist/index.js devices list
```

- 출력에 `pending` 항목과 `RequestID`가 보이면, 해당 ID를 사용해 승인:

```
docker compose exec openclaw-gateway node dist/index.js devices approve
<RequestID>
```

- 그 후 컨테이너를 재시작:

```
docker compose stop
docker compose start
```

- 브라우저 탭에서 F5(새로고침)를 누르면 **정상 채팅 UI**가 등장해야 한다.

8-3. 첫 인사

채팅창에:

> 안녕? 너는 누구니? 앞으로 나를 어떻게 도와줄 수 있어?

정상 답변이 반환되면,

- Docker Desktop
- WSL2 + Ubuntu
- OpenClaw 게이트웨이 컨테이너
- Gemini 온보딩
- 대시보드 접속 + 페어링

까지 **1부에서 필요한 전체 셋업이 완료된 상태**다.

---

9. 셋업 중 자주 나오는 에러 메시지 정리 (Windows/WSL2)

9-1. `The command 'docker' could not be found in this WSL 2 distro.`

의미

- WSL2 Ubuntu 안에서 `docker` 명령을 못 찾는 상태.
- Docker Desktop ↔ WSL2 연동이 안 되어 있거나, 해당 배포판이 WSL2가 아닌 경우.

체크리스트

1. Docker Desktop 실행 여부 확인 (Windows 우측 하단 고래 아이콘).
2. Docker Desktop → Settings → General
   - **Use the WSL 2 based engine** 체크.
3. Docker Desktop → Settings → Resources → WSL Integration
   - **Enable integration with my default WSL distro** 체크.
   - Ubuntu 토글 ON → Apply & Restart.
4. PowerShell에서:

   ```
   wsl -l -v
   ```

   - Ubuntu가 Version 2인지 확인. 1이면:

   ```
   wsl --set-version Ubuntu 2
   ```

5. 이후:

```
wsl --shutdown
```

으로 WSL 재시작 후, Ubuntu 터미널에서 다시 `docker --version` 확인.

9-2. `docker - WSL 2 distro could not be found`

의미

- Docker Desktop이 WSL2 배포판을 찾지 못할 때 주로 발생.
- WSL 설치/업그레이드가 제대로 안 되었거나, Docker에서 WSL2 사용 옵션이 꺼져 있을 수 있음.

체크리스트

1. 관리자 PowerShell에서:

   ```
   wsl --install
   ```

   을 이미 실행했는지, 그리고 재부팅까지 완료했는지 확인.

2. `wsl -l -v`로 실제로 Ubuntu가 목록에 있는지 확인.

3. Docker Desktop → Settings → General / Resources → WSL Integration 설정을 다시 저장.

4. `wsl --shutdown` 후 Docker Desktop 재시작.

9-3. 브라우저에서 `ERR_CONNECTION_REFUSED` / `이 사이트에 연결할 수 없음`

의미

- `http://localhost:3000`에 접속했지만, 해당 포트에서 아무 서비스도 응답하지 않는 상태.
- 게이트웨이 컨테이너가 뜨지 않았거나, 포트 매핑이 잘못되었을 가능성이 큼.

체크리스트

1. Ubuntu에서:

   ```
   cd ~/demo-p1
   docker compose ps
   ```

   - `openclaw-gateway`가 `Up` 상태인지 확인.

2. `docker compose logs -f`로 에러 로그 확인.

3. `.env`의 `OPENCLAW_GATEWAY_PORT`와 `docker-compose.yml`의 `ports` 설정이 일치하는지 확인.

4. 포트 충돌 의심 시, `OPENCLAW_GATEWAY_PORT`를 3001 등으로 바꾸고 다시:

```
docker compose down
docker compose up -d
```

후 `http://localhost:3001`로 접속 시도.

9-4. 대시보드에서 `Unauthorized` / 토큰 관련 에러

의미

- 대시보드와 게이트웨이는 통신하지만, **URL에 전달한 토큰 값이 게이트웨이 토큰과 다름**.

체크리스트

1. `.env`에 설정한 값 확인:

   ```
   OPENCLAW_GATEWAY_TOKEN=...
   ```

2. 브라우저 주소창의 `token=` 값이 위 문자열과 정확히 일치하는지(공백/줄바꿈 포함).
3. `.env`를 수정했다면 반드시:

   ```
   docker compose down
   docker compose up -d
   ```

   으로 컨테이너를 다시 올릴 것.

9-5. `devices list` 했을 때 pending이 안 뜸 / `pairing required` 루프

의미

- 브라우저는 `Pairing Required`를 보여주는데, CLI에서 `devices list`를 해도 `pending`이 비어있는 경우.
- 브라우저가 다른 게이트웨이/토큰에 붙어 있거나, 컨테이너 재시작/설정 문제일 수 있음.

체크리스트

1. 브라우저 주소에서 `token=` 값이 현재 `.env`의 `OPENCLAW_GATEWAY_TOKEN`과 같은지 확인.
2. `devices list`를 실행할 때 **동일한 docker compose 프로젝트 폴더**(예: `~/demo-p1`) 안에서 실행하는지 확인.
3. `docker compose logs`에서 pairing 관련 에러 메시지가 있는지 확인.
4. 브라우저 탭을 모두 닫고, 새로운 탭에서 다시 `http://localhost:3000?token=...`로 접속 후 재시도.

9-6. 온보딩 후 Gemini 관련 `auth_permanent` / `API key invalid` 류 오류

의미

- OpenClaw가 Gemini API 호출을 했지만, 키가 잘못되었거나 비활성화된 상태.
- 또는 온보딩 이후 키를 변경했는데, 기존 설정이 남아 있는 경우.

체크리스트

1. Google AI Studio에서 새 키를 다시 발급받아, 온보딩을 **처음부터 다시 실행**.
2. 온보딩 중 `Paste API key now` 단계에서 새 키를 붙여넣었는지 확인.
3. 컨테이너를 완전히 내렸다가:

   ```
   docker compose down
   docker compose up -d
   ```

   으로 재기동.
4. 여전히 문제가 지속되면, 강의 Q&A에 **로그 일부와 함께** 남겨서 추가 도움 받기.